Chapter 5. Boosting Methods

Predictive Modeling, Lecture 4 (AdaBoost, gradient boosting, XGBoost, LightGBM, CatBoost)

Contents

1 From Averaging to Sequential Correction **2**

2 AdaBoost **2**

2.1 Algorithm 2

2.2 AdaBoost is forward stagewise fitting of the exponential loss 3

3 Gradient Boosting Machines **4**

3.1 Algorithm 4

3.2 Loss functions and their pseudo-residuals 4

3.3 Shrinkage, subsampling and early stopping 4

3.4 Limitations of the standard GBM 5

4 XGBoost: Regularized Second-Order Boosting **5**

5 LightGBM **6**

6 CatBoost **6**

6.1 The curse of dimensionality of categorical variables 7

6.2 Naive target statistics leak the target 7

7 Summary and Practical Rules **8**

1 From Averaging to Sequential Correction

Random forest (Ch. 4)	Gradient boosting (this chapter)
$\hat{f}_{\mathrm{RF}}(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x)$	$F_M(x) = F_0(x) + \sum_{m=1}^{M} \eta\, h_m(x)$
trees learned **independently** on different bootstrap samples	h_m: a shallow tree learning the **error (residual) direction** left by the previous model
averaging reduces the instability of individual trees	η: the **learning rate**, shrinking each correction to prevent overfitting
every tree looks at the same target Y	small corrections ηh_m accumulate **sequentially**, not independently

Random forests mainly reduce variance; boosting mainly reduces bias by adding many weak, shallow learners. This chapter covers *by which principle* boosting learns on tabular data and *under which conditions* to choose it.

2 AdaBoost

Motivation. Boosting builds a strong classifier from many weak ones by focusing each new learner on the examples the current ensemble gets wrong. AdaBoost does this by reweighting, and it turns out to minimize the exponential loss stagewise.

2.1 Algorithm

Algorithm: AdaBoost (labels $y_i \in \{-1, +1\}$)

Initialize $D_1(i) = 1/N$. For $b = 1, \ldots, B$:

1. **Train**: fit a classifier $h_b(x) \in \{-1, +1\}$ using weights D_b.
2. **Evaluate**: weighted error $\epsilon_b = \sum_{i=1}^{N} D_b(i)\, \mathbb{I}\{h_b(x_i) \neq y_i\}$.
3. **Weight**: $\alpha_b = \frac{1}{2} \log \dfrac{1 - \epsilon_b}{\epsilon_b}$.
4. **Update**: $D_{b+1}(i) = \dfrac{D_b(i) \exp\{-\alpha_b y_i h_b(x_i)\}}{Z_b}$, Z_b the normalizer.

Output: $H(x) = \operatorname{sign}\Big(\sum_{b=1}^{B} \alpha_b h_b(x)\Big)$.

Core mechanism: the more an observation was misclassified by previous models, the larger its weight D_{b+1} in the next round ($\exp(+\alpha_b)$ for errors, $\exp(-\alpha_b)$ for correct cases); the better a model (smaller ϵ_b), the larger its say α_b in the final vote. A learner with $\epsilon_b = \frac{1}{2}$ gets $\alpha_b = 0$; one with $\epsilon_b > \frac{1}{2}$ would get a negative weight (it is flipped).

2.2 AdaBoost is forward stagewise fitting of the exponential loss

Theorem: derivation of α_b and the weight update

Let $F_{b-1} = \sum_{s<b} \alpha_s h_s$ and consider the exponential loss $L(y, F) = e^{-yF}$. Choosing (α, h) to minimize $\sum_i \exp\{-y_i(F_{b-1}(x_i) + \alpha h(x_i))\}$ gives: h_b minimizes the weighted error with weights $D_b(i) \propto e^{-y_i F_{b-1}(x_i)}$, and $\alpha_b = \frac{1}{2} \log \frac{1-\epsilon_b}{\epsilon_b}$. The weights evolve exactly by the AdaBoost update.

Proof [Supplement]. With $w_i = e^{-y_i F_{b-1}(x_i)}$, normalized to $D_b(i)$, and $y_i h(x_i) = \pm 1$,

$$\sum_i D_b(i) e^{-\alpha y_i h(x_i)} = e^{-\alpha}(1-\epsilon) + e^{\alpha}\epsilon = e^{-\alpha} + (e^{\alpha} - e^{-\alpha})\epsilon.$$

For any $\alpha > 0$ this is increasing in the weighted error ϵ, so h_b is the weighted-error minimizer. Differentiating in α: $-e^{-\alpha}(1-\epsilon) + e^{\alpha}\epsilon = 0 \iff e^{2\alpha} = (1-\epsilon)/\epsilon$. Finally $w_i^{\text{new}} = w_i e^{-\alpha_b y_i h_b(x_i)}$, which after normalization is the update rule. ■

Theorem: training-error bound

$Z_b = 2\sqrt{\epsilon_b(1-\epsilon_b)}$ and

$$\frac{1}{N}\sum_{i=1}^{N} \mathbb{I}\{H(x_i) \neq y_i\} \leq \prod_{b=1}^{B} Z_b \leq \exp\Big(-2\sum_{b=1}^{B}\gamma_b^2\Big), \qquad \gamma_b = \tfrac{1}{2} - \epsilon_b.$$

Proof [Supplement]. Plugging α_b into $Z_b = e^{-\alpha_b}(1-\epsilon_b) + e^{\alpha_b}\epsilon_b$ gives $2\sqrt{\epsilon_b(1-\epsilon_b)}$. Unrolling the update, $D_{B+1}(i) = \frac{e^{-y_i F_B(x_i)}}{N\prod_b Z_b}$, and these sum to 1, so $\frac{1}{N}\sum_i e^{-y_i F_B(x_i)} = \prod_b Z_b$. Since $\mathbb{I}\{H(x_i) \neq y_i\} \leq e^{-y_i F_B(x_i)}$, the first inequality follows. Finally $2\sqrt{\epsilon(1-\epsilon)} = \sqrt{1-4\gamma^2} \leq e^{-2\gamma^2}$. ■

So if every weak learner is slightly better than chance ($\gamma_b \geq \gamma > 0$) the training error falls exponentially fast. The population minimizer of the exponential loss is $f^\star(x) = \frac{1}{2}\log\frac{P(Y=1|x)}{P(Y=-1|x)}$ (minimize $pe^{-f} + (1-p)e^{f}$), so AdaBoost estimates half the log-odds.

Example: AdaBoost on a toy case (classic illustration)

Ten points in the plane with decision stumps. Round 1: a vertical stump misclassifies three + points, $\epsilon_1 = 0.30$, $\alpha_1 = \frac{1}{2}\log(0.7/0.3) = 0.42$; the three misclassified points are enlarged in D_2. Round 2: another vertical stump, $\epsilon_2 = 0.21$, $\alpha_2 \approx 0.65$; three − points it misclassifies grow in D_3. Round 3: a horizontal stump, $\epsilon_3 = 0.14$, $\alpha_3 \approx 0.92$. The final classifier

$$H_{\text{final}} = \text{sign}(0.42\, h_1 + 0.65\, h_2 + 0.92\, h_3)$$

is a staircase boundary that no single stump can represent. (With the formula, 0.21 and 0.14 give 0.66 and 0.91; the figures round the weighted errors.)

3 Gradient Boosting Machines

3.1 Algorithm

Algorithm: GBM

Initialize $F_0(x) = \arg\min_\gamma \sum_{i=1}^N L(y_i, \gamma)$. For $b = 1, \dots, B$:

1. **Gradient**: pseudo-residuals $r_{ib} = -\Big[\dfrac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\Big]_{F=F_{b-1}}$.
2. **Fit**: fit a regression tree $h_b(x)$ (leaves R_{jb}) to the targets r_{ib}; optionally set each leaf value by a line search $\gamma_{jb} = \arg\min_\gamma \sum_{x_i \in R_{jb}} L(y_i, F_{b-1}(x_i) + \gamma)$.
3. **Update**: $F_b(x) = F_{b-1}(x) + \eta\, h_b(x)$.

Output: $F_B(x) = F_0(x) + \sum_{b=1}^B \eta\, h_b(x)$.

Core mechanism: if the loss L is squared error, the gradient r_{ib} coincides exactly with the ordinary residual $y_i - F_{b-1}(x_i)$. Unlike AdaBoost, **any differentiable loss** can be used. It is the principle of **gradient descent in function space**: the vector $(F(x_1), \dots, F(x_N))$ is moved in the negative gradient direction, and the tree h_b is the best approximation of that direction that can also be evaluated at new x.

3.2 Loss functions and their pseudo-residuals

Loss	$\Psi(y, f)$	$-\partial\Psi/\partial f$	F_0
squared (L_2)	$\frac{1}{2}(y-f)^2$	$y - f$	mean
absolute (L_1)	$\lvert y - f\rvert$	$\operatorname{sign}(y-f)$	median
Huber	$\frac{1}{2}(y-f)^2$ if $\lvert y-f\rvert \le \delta$; $\delta(\lvert y-f\rvert - \delta/2)$ otherwise	$y - f$ or $\delta\operatorname{sign}(y-f)$	robust center
quantile	$(1-\alpha)\lvert y-f\rvert$ if $y - f \le 0$; $\alpha\lvert y-f\rvert$ if $y - f > 0$	α or $-(1-\alpha)$	α-quantile
Bernoulli	$\log(1 + e^{-2\tilde{y}f})$, $\tilde{y} \in \{-1, 1\}$	$\dfrac{2\tilde{y}}{1 + e^{2\tilde{y}f}}$	$\frac{1}{2}\log\frac{\bar{p}}{1-\bar{p}}$
exponential (AdaBoost)	$e^{-\tilde{y}f}$	$\tilde{y}e^{-\tilde{y}f}$	$\frac{1}{2}\log\frac{\bar{p}}{1-\bar{p}}$

In the Bernoulli loss, f is half the log-odds ($p = 1/(1 + e^{-2f})$); with f equal to the log-odds the same loss is the usual logistic log loss with pseudo-residual $y - p$, $y \in \{0, 1\}$. With absolute or Huber loss a few large residuals cannot dominate the trees (Ch. 2).

3.3 Shrinkage, subsampling and early stopping

Small learning rates ($\eta = 0.01$–0.1) need more trees but generalize better: each tree only partially corrects the residual, which acts like regularization along the path. **Stochastic gradient boosting** fits each tree on a random subsample of rows (and/or columns), which reduces correlation and variance. The number of trees B is chosen by **early stopping** on a validation set: stop when validation loss has not improved for a given number of rounds. Learning rate, number of trees, tree depth (interaction order), subsampling and early stopping control the *same* overfitting problem from different places.

3.4 Limitations of the standard GBM

Standard GBM predicts very well but has practical limits: with too many trees it can overfit; it contains no explicit regularization of tree complexity; sequential tree fitting can be computationally inefficient. Modern implementations address these.

4 XGBoost: Regularized Second-Order Boosting

Motivation. Gradient boosting fits each tree to the negative gradient. XGBoost uses a second-order Taylor expansion of the loss, which gives closed-form leaf values and a split gain that includes explicit regularization.

XGBoost improves both the *mathematical optimization* and the *system computation*:

- **Regularization**: GBM has no built-in complexity penalty; XGBoost adds L_1/L_2 penalties of the leaf weights and a per-leaf cost to the objective.
- **Second-order optimization**: GBM follows only the first derivative (gradient) and can converge slowly; XGBoost uses a second-order Taylor expansion, i.e. the curvature (Hessian), to reach the optimum faster and more precisely.

Theorem: optimal leaf weights and split gain

At round t the new tree $f_t(x) = w_{q(x)}$ (leaf index $q(x)$, T leaves) minimizes

$$\mathcal{L}^{(t)} = \sum_i L(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \gamma T + \tfrac{1}{2}\lambda \sum_{j=1}^{T} w_j^2.$$

With $g_i = \partial_{\hat{y}} L(y_i, \hat{y}_i^{(t-1)})$, $h_i = \partial^2_{\hat{y}} L(y_i, \hat{y}_i^{(t-1)})$, $G_j = \sum_{i \in I_j} g_i$, $H_j = \sum_{i \in I_j} h_i$, the second-order approximation gives

$$w_j^\star = -\frac{G_j}{H_j + \lambda}, \qquad \tilde{\mathcal{L}}^\star = -\frac{1}{2} \sum_{j=1}^{T} \frac{G_j^2}{H_j + \lambda} + \gamma T,$$

and the gain of splitting a leaf into L and R is

$$\text{Gain} = \frac{1}{2}\Big[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}\Big] - \gamma.$$

Proof [Supplement]. Taylor: $L(y_i, \hat{y} + f) \approx L(y_i, \hat{y}) + g_i f + \frac{1}{2} h_i f^2$. Dropping constants and grouping by leaves, $\tilde{\mathcal{L}} = \sum_j \{G_j w_j + \frac{1}{2}(H_j + \lambda) w_j^2\} + \gamma T$, a separate quadratic in each w_j; its minimizer is $-G_j/(H_j + \lambda)$ with value $-\frac{1}{2} G_j^2/(H_j + \lambda)$. The gain is the objective of one leaf minus that of two leaves, minus the extra leaf cost γ. ■

Example: what $w^\star$ is for familiar losses

Squared loss $\frac{1}{2}(y - \hat{y})^2$: $g_i = \hat{y}_i - y_i$, $h_i = 1$, so $w_j^\star = \sum_{i \in I_j} (y_i - \hat{y}_i)/(n_j + \lambda)$, a *shrunken* mean residual (Ridge on leaf values). Logistic loss on the log-odds scale: $g_i = p_i - y_i$, $h_i = p_i(1 - p_i)$, so $w_j^\star$ is one Newton step for the leaf. The constraint `min_child_weight` bounds H_j from below; γ prunes splits whose gain is too small.

On the system side: sparsity-aware split finding learns a default direction for missing values, approximate (quantile-sketch) split candidates, column blocks and cache-aware parallel computation.

5 LightGBM

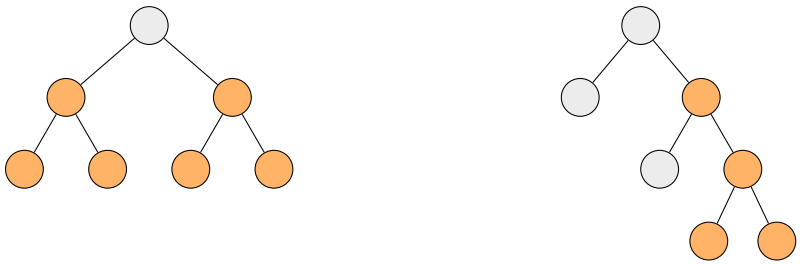
LightGBM is a highly efficient gradient-boosting implementation optimized for large and high-dimensional data:

- **Histogram-based learning**: continuous variables are bucketed into a small number of bins, drastically reducing computation and memory.
- **Leaf-wise tree growth**: the tree is not grown symmetrically; only the leaf whose split reduces loss most is expanded, deeply and selectively.
- **Efficient computation**: combining the two cuts training time and memory dramatically.

Aspect	Standard GBM	LightGBM
split search	pre-sorts every continuous value and scans all of them; bottleneck on large data	integer bins (histogram) cut computation and memory
tree growth	level-wise: grows all nodes of a level symmetrically, even unnecessary ones	leaf-wise: expands only the node that reduces the error most

Derivation: why histograms are cheap

With k bins (default 255), building a node's histogram of (G, H) sums costs $O(\#\text{rows in node})$ per feature, and finding the best split costs only $O(k)$ instead of $O(\#\text{rows})$. Moreover a child's histogram is the parent's minus the sibling's (*histogram subtraction*), so only the smaller child needs to be built. Coarse bins act as mild regularization.



level-wise (GBM, XGBoost default) leaf-wise (LightGBM)

Figure 1: Orange nodes are created by the most recent splits. Level-wise growth splits every node of a depth before going deeper; leaf-wise growth always splits the single leaf with the largest loss reduction, giving deep, unbalanced trees for the same number of leaves.

Remark: leaf-wise growth and overfitting

For the same number of leaves, leaf-wise growth attains lower training loss than level-wise growth, but it produces deep, unbalanced trees that overfit small data; control `num_leaves`, `max_depth`, `min_data_in_leaf`. LightGBM also uses gradient-based one-side sampling (GOSS: keep rows with large $|g_i|$, subsample the rest with reweighting) and exclusive feature bundling (EFB: merge sparse features that are rarely nonzero simultaneously).

6 CatBoost

Motivation. Target statistics for categorical features and ordinary gradient boosting both reuse a sample's own label, which leaks information and biases the fit. CatBoost's ordered scheme computes

everything for a sample from earlier samples only.

6.1 The curse of dimensionality of categorical variables

- GBM: hundreds of categories require one-hot encoding into hundreds of dummy columns; memory and trees become inefficient.
- CatBoost: however many categories, each is replaced *within a single column* by the proportion of target = 1 (target statistics), preventing the explosion of dimensions.

Example: one-hot (GBM) versus target statistics (CatBoost)

Category	Share of target = 1 in that category	CatBoost value (one column)
A	80 of 100 (80%)	$\to 0.80$
B	10 of 40 (25%)	$\to 0.25$
C	100 of 200 (50%)	$\to 0.50$
D	5 of 50 (10%)	$\to 0.10$

Instead of four new columns for A, B, C, D, one continuous column $0.80, 0.25, \ldots$ replaces them.

6.2 Naive target statistics leak the target

Theorem: target leakage of in-sample target encoding

If each row's encoding uses its own target, $\hat{x}_i = \frac{1}{n_{c_i}} \sum_{j:c_j=c_i} y_j$, then for a category observed once the encoded value equals y_i. In the extreme of all categories unique (e.g. an ID), the encoded feature equals the target on the training data and a single split fits it perfectly, while on new data the feature carries no information.

More generally, $\mathbb{E}[\hat{x}_i \mid y_i]$ depends on y_i through the term y_i/n_{c_i}, so the distribution of the encoded feature differs between training rows (which saw their own target) and test rows (*target shift*). CatBoost fixes this in two steps:

- **smoothing with a prior**: $\hat{x}_i = \dfrac{\sum_{j \in \mathcal{P}_i} \mathbb{I}(c_j = c_i)\, y_j + a\, \bar{p}}{\sum_{j \in \mathcal{P}_i} \mathbb{I}(c_j = c_i) + a}$, shrinking rare categories toward the global rate $\bar{p}$;
- **ordered target statistics**: draw a random permutation σ and let $\mathcal{P}_i = \{j : \sigma(j) < \sigma(i)\}$, i.e. each row is encoded using only rows that come *before* it, as in an online stream. A row's own target never enters its encoding, mimicking the test situation.

The same idea, **ordered boosting**, computes the residuals of a row with a model that has not seen that row, removing the "prediction shift" of standard GBM. CatBoost also uses symmetric (oblivious) trees, the same split at every node of a level, which are fast to evaluate and regularize the trees.

7 Summary and Practical Rules

Algorithm	Direction of development
XGBoost	explicit regularization controls overfitting; second-order information (Hessian) gives precise optimization
LightGBM	histograms and leaf-wise growth drastically reduce computation on large numerical/sparse data
CatBoost	ordered target statistics handle leakage and instability of categorical features

Rules for using boosting.

- Boosting adds shallow trees sequentially, updating the predictor in the direction that most reduces the current loss.
- Learning rate, number of iterations, tree complexity, subsampling and early stopping control the same overfitting problem from different places.
- Choose a model not by its name or one best score but by the performance and error structure repeated under a fixed validation procedure (time and group structure of Ch. 1 still apply; early stopping uses a validation fold, not the test set).